Build and Flash

1. Overview

Arduino IDE's Compile (Verify) and Flash (Upload) are one-click operations. Click **Verify (Compile)** button to compile `.ino` source code into `.bin` firmware file runnable on ESP32-S3; Click **Upload** button to first compile then write firmware to chip's Flash memory via serial port, chip auto-resets and runs new program after writing completes.

This tutorial organizes four steps: Project Save -> Compile -> Flash -> Serial Monitor, each step corresponds to one toolbar button.

2. Prerequisites

Before proceeding, ensure the following conditions are met:

Condition	Requirement	Verification Method
IDE installed	Arduino IDE 2.x installed	Double-click desktop shortcut starts normally
ESP32 platform installed	`esp32 by Espressif Systems` shows `INSTALLED` in Boards Manager	Left toolbar -> Boards Manager -> Search "ESP32" to confirm
Board selected	`ESP32S3 Dev Module` selected on top toolbar	Bottom status bar shows `ESP32S3 Dev Module`
Driver installed	CH340 driver installed	No yellow exclamation in Device Manager -> Ports
Project saved	Current window has `.ino` file	Title bar shows project name not "Untitled"

3. Compile (Verify)

Arduino IDE's **Verify (Compile)** button compiles source code into `.bin` firmware file without flashing, used to check for syntax and linking errors.

3.1 Steps

1. **Open any `.ino` project**
2. **Click  Verify (Compile) icon on top toolbar**, or press `Ctrl + R`
3. **Observe bottom Output panel**, output similar to:

```
Sketch uses 319136 bytes (24%) of program storage space. Maximum is 1310720 bytes.
Global variables use 22088 bytes (6%) of dynamic memory, leaving 305592 bytes for
local variables. Maximum is 327680 bytes.
```

Field	Meaning
`program storage space`	Flash firmware storage usage; this program uses 319136 bytes, partition total limit 1310720 bytes (1.25MB), utilization 24%
`Maximum is 327680 bytes`	On-chip DRAM (internal SRAM) total max capacity, 320KB total, not part of Flash space
`Global variables`	Global static variables occupy DRAM space; 22088 bytes total, 6% of memory capacity
`dynamic memory`	Same as on-chip SRAM/DRAM, used for global variables, task stacks, local variables, heap dynamic memory, remaining 305592 bytes freely available

4. **If no errors**, bottom shows `Done compiling` prompt, compilation complete.
5. **If errors**, Output panel lists error line numbers and causes in red text:

- Double-click error line to jump to corresponding code location
- Common errors: missing semicolons, spelling mistakes, library not installed, macro undefined

> First compilation takes about 1~3 minutes (including linking and generating `.bin`), incremental compilation (only modified a few lines) usually just a few seconds.

3.2 Compilation Output

After compilation completes, firmware file is generated in temp directory (invisible in IDE), but you can export `.bin` file to project directory via menu **Sketch -> Export compiled Binary**. Exported files include:

File Suffix	Description
`.bin`	Main firmware (copied to Flash APP partition)
`.bin.bootloader`	Bootloader (embedded in `.bin`, usually not needed separately)

---

4. Flash (Upload)

Flashing writes compiled `.bin` firmware to ESP32-S3's Flash memory via COM port. After clicking **Upload**, IDE automatically completes: Compile -> Connect to chip -> Erase partition -> Write firmware -> Verify -> Reset and run, no manual operation needed.

4.1 Steps

1. **Connect ESP32-S3 development board to computer with USB cable**
2. **Confirm correct port selected on top toolbar** (e.g., `COM6`)
3. **Click  Upload on top toolbar**, or press `Ctrl + U`
4. **Observe bottom Output panel for flashing progress**:

```
esptool v5.2.0
Serial port COM6:
Connecting....
Connected to ESP32-S3 on COM6:
Chip type:          ESP32-S3 (QFN56) (revision v0.2)
Features:           Wi-Fi, BT 5 (LE), Dual Core + LP Core, 240MHz, Embedded PSRAM 8MB
(AP_3v3)
Crystal frequency:  40MHz
MAC:                a4:cb:8f:c6:b8:64
Uploading stub...
Running stub...
Stub running...
Configuring flash size...
Flash will be erased from 0x00010000 to 0x0005ffff...
Compressed 261937 bytes to 151335...
Writing at 0x00010000... (100%)
Verifying written data...
Hash of data verified.
```

```
Hard resetting via RTS pin...
```

Log	Meaning
`Serial port COM6`	Currently used serial port
`Connecting....`	Attempting handshake with chip bootloader
`Chip is ESP32-S3`	Chip model identified successfully
`Crystal is 40MHz`	Crystal frequency identified
`Uploading stub...`	Uploading flashing stub (mini flashing program running in RAM)
`Compressed xxx bytes to xxx`	Firmware size (after/before compression)
`Writing at 0x00010000... (100%)`	Firmware write Flash address + progress
`Hard resetting via RTS pin...`	Pull down RTS pin to reset chip, new firmware starts running

5. **After flashing completes**, chip auto-resets, new program starts running.

4.2 Flashing Failed — Manually Enter Download Mode

Some ESP32-S3 development boards cannot auto-enter download mode for flashing (USB-to-serial chip's RTS/DTR pins not connected to chip EN/IO0), manual operation needed:

Operation flow: Hold BOOT button -> Press RST button (reset) once -> Release BOOT button.

```
Step 1:  Hold BOOT button (don't release)
Step 2:  Press RST button once (press and release)
Step 3:  Release BOOT button
         ↑ Chip enters download mode, IDE auto-starts flashing after detection
```

> To quickly determine if manual download mode is required: If you see an infinite loop of dots on the "Connecting..." line in the Output panel, it means the chip has not entered download mode.

4.3 Flashing Address Description

Arduino IDE defaults to flashing ESP32-S3's Flash, partition addresses defined by partition table (under default partition table):

```
Flash Layout (Default Partition Table)
┌──────────┬──────────┬──────────┬──────────┬──────────┐
|Bootloader|Partition |  NVS     |  app0    |   SPIFFS  |
| 32KB     | Table    |  24KB    |  3.14MB  |   Remaining|
|0x0000    |0x8000    |0x9000    |0x10000   |           |
└──────────┴──────────┴──────────┴──────────┴──────────┘
```

Partition	Start Address	Size	Description
Bootloader	`0x0000`	32KB	Secondary bootloader, runs first after power-on
Partition Table	`0x8000`	4KB	Partition table metadata, describes partition structure
NVS	`0x9000`	24KB	Non-volatile storage (Wi-Fi calibration data, etc.)
app0	`0x10000`	~3.14MB	APP partition, firmware flashed here

> Flashing log `Writing at 0x00010000` corresponds to app0 partition start address.

---

5. Serial Monitor

After flashing, use **Serial Monitor** to view chip serial log output, verifying program runs as expected.

5.1 Steps

1. **Click  Serial Monitor icon on top toolbar**, or press `Ctrl + Shift + M`
2. **Confirm baud rate at bottom of monitor panel matches `Serial.begin()` in code**

   > All examples in this tutorial use **115200**.

3. **Observe serial output**
4. If chip didn't auto-reset, press **RST reset button** on development board to let program run from beginning, monitor will re-output `setup()` startup logs.

5.2 Quick Operations

Operation	Description
Click input box at bottom of monitor panel + Enter	Send a line of text to chip
Check `Autoscroll`	New logs auto-scroll to bottom
Check `Show timestamp`	Add timestamp before each log line
Baud rate dropdown at bottom right	Switch baud rate (must match code)
X button	Close monitor panel

---

6. Serial Plotter

Arduino IDE has built-in **Serial Plotter**, plots `Serial.print()` output values as real-time waveform curves, ideal for observing ADC sample values, sensor readings and other continuously changing data.

6.1 Usage

1. In code, separate multiple values with commas:

```
void setup(){
    Serial.begin(115200);
}
void loop() {
    int sensor1 = analogRead(A0);
    int sensor2 = analogRead(A1);
    Serial.print(sensor1);
    Serial.print(", ");
    Serial.println(sensor2);        // Comma separated -> two curves
    delay(50);
}
```

2. Click toolbar  **Serial Plotter icon**
3. Waveform window auto-pops up, horizontal axis is time, vertical axis is value.

Rule	Description
Multiple values on one line separated by `,`	Plot multiple curves in different colors
Each line ends with `\n`	One line = one sample point
Vertical axis auto-scales	Dynamically adjusts based on current visible range
`Ctrl + Shift + L`	Shortcut to open plotter

7. Flash Firmware Tool

7.1 What is Flash Firmware Tool

ESP Flash Download Tool is Espressif's official Windows GUI flashing tool, suitable for **not installing Arduino IDE / ESP-IDF environment, only flashing pre-compiled .bin firmware** scenarios.

Comparison	Arduino IDE Upload	Flash Firmware Tool
Need source code	Yes (.ino file)	No (only .bin firmware)
Need Arduino IDE	Yes	No
Batch flashing	Inconvenient (open projects one by one)	Convenient (repeat click START)
Flash multiple .bin	Not supported	Supported (bootloader + partition + app)

Comparison	Arduino IDE Upload	Flash Firmware Tool
Merge firmware	Not supported	Supported (one-click merge to target .bin)
Use case	Development debugging	Mass production flashing, firmware distribution, factory restore

> When you can't install Arduino IDE (e.g., someone else's computer, workshop station), or need to flash same firmware to multiple boards, Flash Firmware Tool is the fastest way.

7.2 Installation

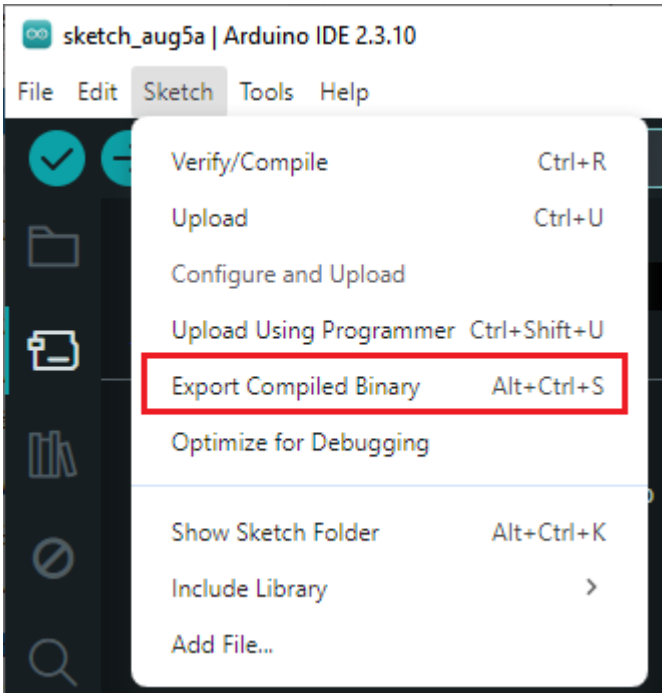
Simply extract `6. Software Tools -> Firmware Tool -> flash_download_tool.zip` (ESP Flash Download Tool Official Download), no installation needed, double-click `flash_download_tool_x.x.x.exe` to use.

7.3 Export .bin Firmware

Before using flashing tool, need to export compiled `.bin` firmware from Arduino IDE. Arduino IDE's Upload operation auto-compiles and flashes in background, but compilation output is in temp directory, not auto-retained.

Export steps:

1. Open project in Arduino IDE
2. Select board (`ESP32S3 Dev Module`), ensure project compiles successfully
3. Click menu **Sketch -> Export compiled Binary**



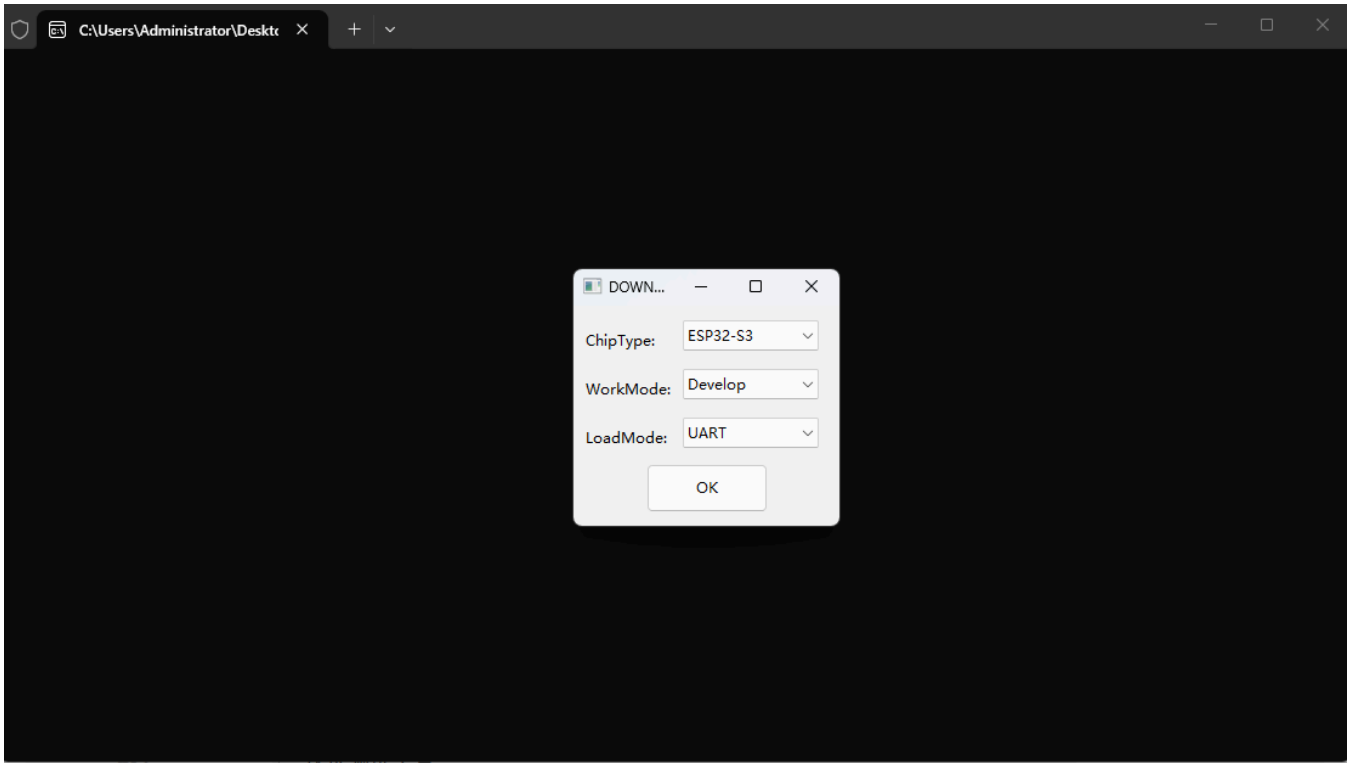
7.4 Steps

Step 1: Launch Tool

After extracting `flash_download_tool.zip`, enter extraction directory and double-click `flash_download_tool_x.x.x.exe` -> Select chip model `ESP32-S3` -> Confirm.

Step 2: Configure Flashing Files

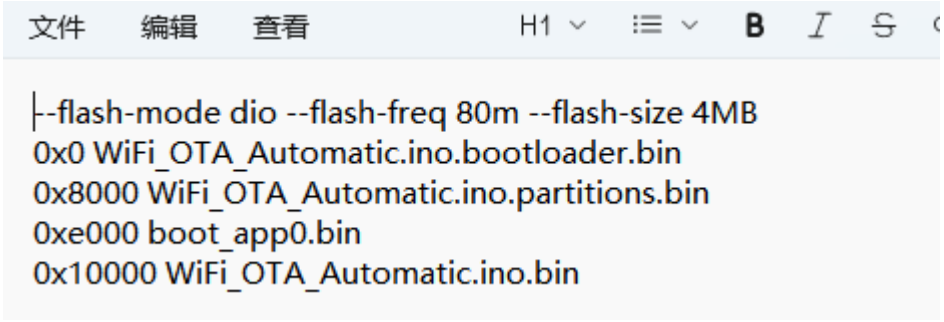
Add firmware file on flashing interface, fill in corresponding flashing address:



File	Address	Description
`project.ino.bootloader.bin`	`0x0`	Secondary bootloader
`project.ino.partitions.bin`	`0x8000`	Partition table
`project.ino.bin`	`0x10000`	Main application firmware

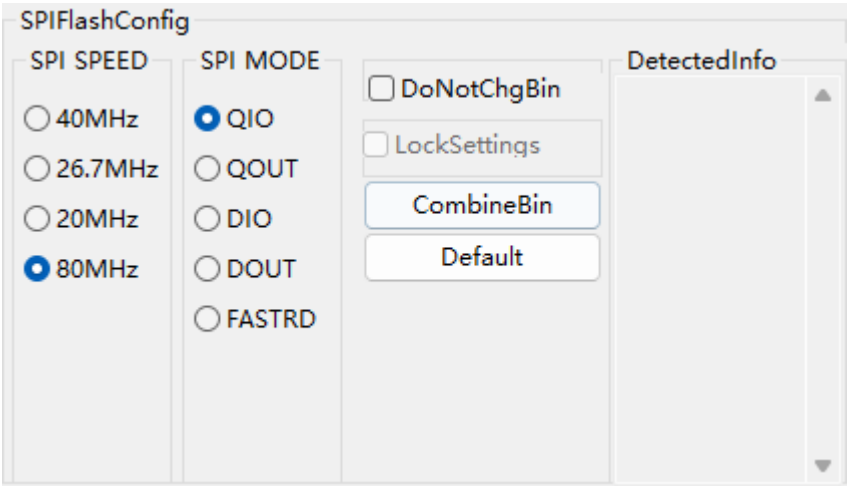
> File path cannot contain Chinese or spaces, otherwise tool parsing fails. Recommend copying `.bin` files to English path (e.g., `D:\firmware\`) before adding.

Addressing method: After exporting binary file, enter folder build->esp32.esp32.esp32s3->flash_args, open with notepad, to get files and corresponding locations.



Check checkbox before each file to activate that flashing item.

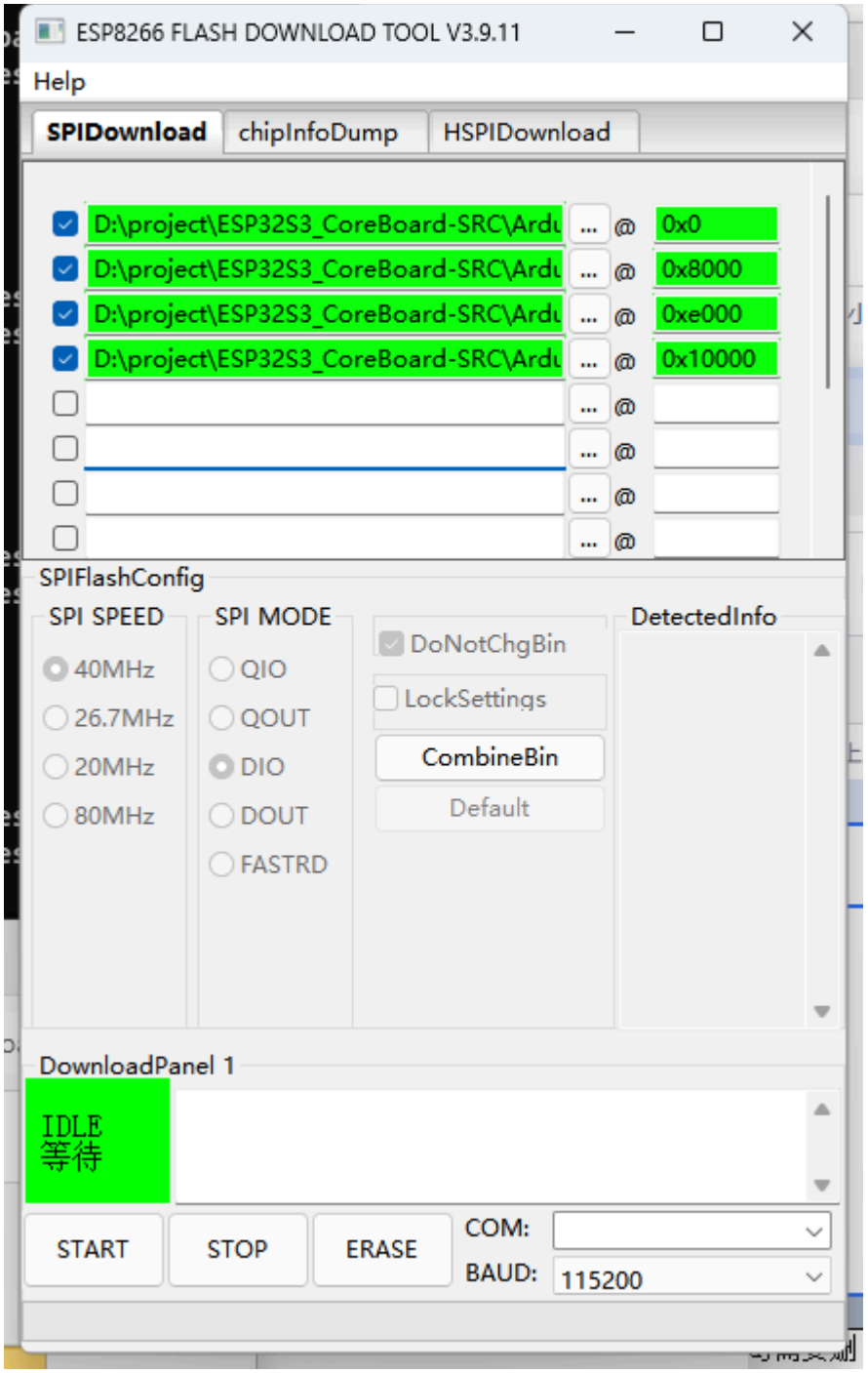
Step 3: Configure Flashing Parameters



Parameter	Recommended Value	Description
SPI SPEED	80MHz	SPI Flash clock frequency
SPI MODE	QIO	Flash communication mode (Quad I/O)
FLASH SIZE	16Mbit (N16R8)	Select according to actual Flash capacity
BAUD	1152000	Flashing baud rate

Step 4: Connect and Flash

1. Select development board corresponding **COM Port** (When using Flash firmware tool, serial port used must not be occupied, otherwise flashing will fail)



2. Set baud rate to **1152000**
3. Click **START** to start flashing
4. When progress bar fills, shows **FINISH**

Step 5: Manually Enter Download Mode

If tool keeps showing `Waiting for power-on sync`, manually let chip enter download mode:

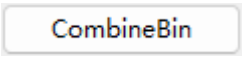
1. **Hold BOOT button**
2. **Press RST button once** (reset)
3. **Release BOOT button**

> Some development boards need this operation, especially after flashing and running other firmware.

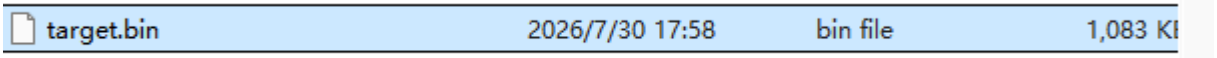
7.5 Merge Firmware Flashing

When needing to merge multiple `.bin` files into one complete firmware (for mass production flashing or distribution), use tool's merge function:

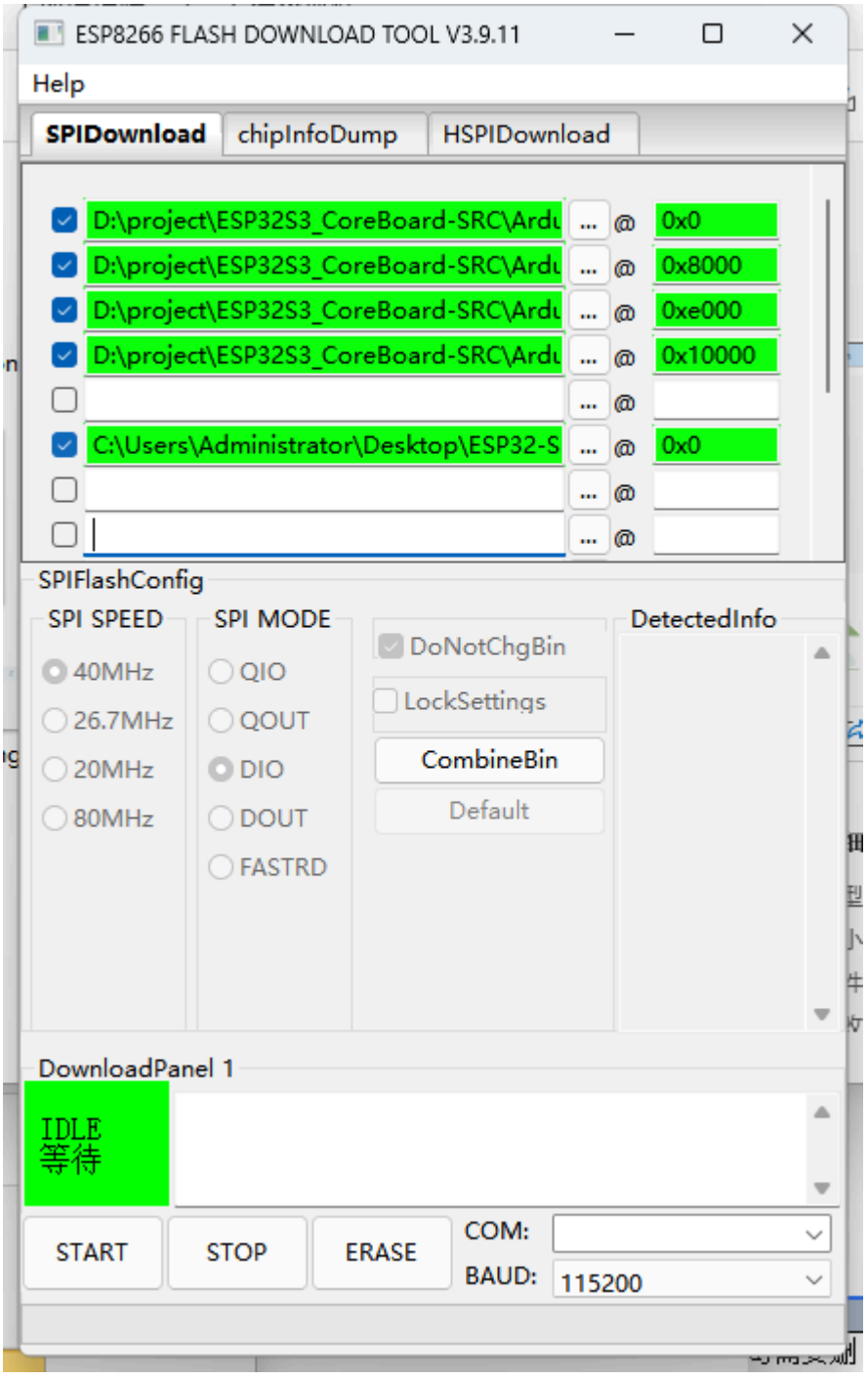
Click tool interface's **Merge** button -> Pop up merge config dialog -> Confirm merge files and addresses -> Generate merged `.bin` file.



A popup will appear, generating a new file `.bin`



Rename merged file (example `hello_esp32s3_merged.bin`):



> Merged `.bin` file only needs to fill in **address** `0x0`, check and flash. One file covers bootloader + partition table + application firmware all content.

---

8. FAQ

Phenomenon	Cause	Solution
Compilation error "avr-gcc: command not found"	ESP32-S3 board not selected (IDE defaults to AVR toolchain)	Search and select `ESP32S3 Dev Module` in top toolbar board dropdown
Compilation error "xxx.h: No such file or directory"	Third-party library not installed	Left toolbar -> Library Manager -> Search and install corresponding library
Upload failed "Timed out waiting for packet header"	Chip not in download mode	Manually enter download mode: **Hold BOOT -> Press EN -> Release BOOT**
Upload failed "No serial data received"	Wrong serial port / serial port occupied by other tools	Close other serial tools (serial assistant, putty), confirm correct COM port
Upload failed "Wrong boot mode detected"	Chip in running state not download state	Hold BOOT -> Press EN -> Observe chip LED turns off after EN release -> Release BOOT
Upload failed "A fatal error occurred: Failed to connect to ESP32-S3"	USB cable only powers, no data transfer	Replace cable; or update CH340 driver to latest version
Upload succeeded but program doesn't run	Chip didn't auto-reset	Press EN (RST) button to manually reset after flashing
Serial Monitor shows garbled text	Baud rate mismatch / IDE monitor baud rate setting error	Confirm `Serial.begin()` parameter in code matches monitor bottom-right baud rate (115200 in this tutorial)
Serial Monitor has no output	USB CDC On Boot not enabled / wrong port selected	Tools -> USB CDC On Boot -> Enabled; confirm correct COM port selected
Serial Monitor shows "????" or unreadable characters	Encoding issue	`Serial.print()` in code uses English only; monitor encoding defaults to UTF-8
IDE doesn't detect COM port	Driver not installed / USB cable doesn't support data	Install CH340 driver; replace with data-capable cable
Flashing stuck mid-process	USB poor contact or EMI	Replug USB cable; use USB port on motherboard back (front panel may have insufficient power)

Phenomenon	Cause	Solution
Incremental compilation still slow	First compilation didn't retain intermediate products	Normal — incremental compilation fast within same project, full rebuild needed across projects or after header file modifications
Flash firmware tool keeps "Waiting for power-on sync"	Chip not in download mode	Manual: Hold BOOT -> Press EN -> Release BOOT, then click START
Flash firmware tool file path error	Path contains Chinese or spaces	Move `.bin` files to English path (e.g., `D:\firmware\`)
Flash firmware tool program doesn't run after flashing	FLASH SIZE wrong / doNotChgBin not checked	For N16R8 model select **16Mbit** Flash; confirm SPI MODE is QIO
Export .bin prompts "Please select a board first"	Board not selected	Confirm `ESP32S3 Dev Module` selected in top toolbar
Export compiled Binary menu is grayed out	Current project never successfully compiled	First click Verify (checkmark ✓) to compile successfully once, then export